Energy-Aware Compiler: Static Analysis and Optimized Code Generation

Architecture Overview

Input: Source Code

1. Lexer (Tokenization)
2. Parser (Syntax Analysis)
3. Abstract Syntax Tree (AST)
4. Semantic Analyzer
5. Optimizer
6. Energy Analysis Engine
7. Code Generator

Output: Compiler Output + Energy Report

Tech Stack

Part	Technology
Compiler	C++
Lexer	Hand-written C++ (character scanning, no regex library needed)
Parser	Recursive Descent Parser (C++)
AST	C++ Classes / Structs
Symbol Table	std::unordered_map
Optimizer	C++ (IR-to-IR passes)
Energy Analyzer	C++ (pass over optimized IR)
Code Generation	C++ Code Generator or Three Address Code
Report	HTML / PDF / Text
Version Control	GitHub

Initial Things to Finalize

Token format (what the lexer outputs) — e.g. {type: "KEYWORD", value: "repeat", line: 1}

AST format (what the parser outputs) — the node shape for Let, Repeat, Print, Expr

IR format (what semantic analysis/lowering outputs) — the flat instruction list shape, e.g. {op: "PRINT", arg: "x", line: 3}

Project Workflow Summary (C++ Implementation)

EcoLang Source Code (.eco)

1. Lexical Analysis (Lexer)

Reads the EcoLang source code character by character and converts it into a stream of tokens.

```
Input:
let x = 5

Output:
KEYWORD(let)
IDENTIFIER(x)
ASSIGN(=)
NUMBER(5)
```

Files: lexer.cpp, token.h

2. Syntax Analysis (Parser)

Reads the token stream and checks whether the program follows EcoLang grammar. If the syntax is valid, it builds an Abstract Syntax Tree (AST), which represents the structure of the program.

```
let x = 5
print x

↓

Program
├── LetStatement
│   ├── x
│   └── 5
└── PrintStatement
    └── x
```

Files: parser.cpp, ast.h

Implementation: Recursive Descent Parser; AST implemented using C++ classes

3. Semantic Analysis

Traverses the AST and checks whether the program is logically correct.

Checks performed:

- Variable declaration before use
- Variable redeclaration
- Type mismatches
- Invalid operations

```
print y

↓

Semantic Error:
Variable 'y' not declared.
```

Files: semantic.cpp, semantic.h

Data Structure: Symbol Table (std::unordered_map)

4. Intermediate Representation (IR) Generation

Converts the AST into a simplified Intermediate Representation (IR), which is easier to analyze and optimize.

```
let x = 5
print x

↓

STORE x 5
PRINT x
```

Files: ir.cpp

Implementation: IR instructions stored using C++ structs/classes, held in a std::vector

5. Code Optimization

Analyzes the IR to improve efficiency before energy analysis and final code generation. This step now runs before the Energy Analysis Engine — an earlier draft of this document had these two steps in the opposite order in some sections, which contradicted the project's own goal of scoring the optimized program.

Optimizations:

- Constant Folding
- Loop-Invariant Detection
- Repeated Calculation Detection
- Expensive I/O Detection

Generates optimization suggestions such as:

- Reduce print statements inside loops
- Cache repeated calculations
- Remove unused variables

Files: optimizer.cpp

6. Energy Analysis Engine

Traverses the optimized IR and estimates the computational cost of the program. Each instruction is assigned an energy weight.

Example Cost Table:

- Assignment = 1 unit
- Arithmetic = 1 unit
- Loop = 1 unit
- Print = 5 units

Calculates:

- Total operations
- Estimated energy units
- Estimated energy (Joules)
- Time complexity
- Operation-wise energy breakdown

Files: energy.cpp

7. Code Generation

Converts the optimized IR into executable output.

Possible outputs:

- C++ Code (recommended)
- Three-Address Code (TAC)
- Pseudo Assembly

```
EcoLang

repeat 3
    print "Hello"
end

↓

Generated C++

for (int i = 0; i < 3; i++)
{
    std::cout << "Hello";
}
```

Files: codegen.cpp

8. Final Report Generation

Displays the compiler output along with the energy analysis.

```
----------------------------------------
Compilation Successful

Estimated Operations   : 2003
Estimated Energy Units : 6003
Estimated Energy       : 0.0006 J
Time Complexity        : O(n)

Energy Breakdown
Assignments            : 2
Arithmetic Operations  : 1
Loop Operations        : 1000
Print Statements       : 1000

Optimization Suggestions
✓ Reduce print statements inside loops
✓ Cache repeated calculations
✓ Remove unused variables
----------------------------------------
```

Files: report.cpp

Overall Compiler Pipeline

```
EcoLang Source Code (.eco)
        ↓
Lexical Analysis (lexer.cpp)
        ↓
Token Stream
        ↓
```

```
Syntax Analysis (parser.cpp)
        ↓
Abstract Syntax Tree (AST)
        ↓
Semantic Analysis (semantic.cpp)
        ↓
Intermediate Representation (ir.cpp)
        ↓
Code Optimization (optimizer.cpp)
        ↓
Energy Analysis Engine (energy.cpp)
        ↓
Code Generation (codegen.cpp)
        ↓
Final Energy Report (report.cpp)
```

C++ Project Structure

```
EcoLang/
├── include/
│   ├── token.h
│   ├── lexer.h
│   ├── parser.h
│   ├── ast.h
│   ├── semantic.h
│   ├── ir.h
│   ├── optimizer.h
│   ├── energy.h
│   ├── codegen.h
│   └── report.h
│
├── src/
│   ├── lexer.cpp
│   ├── parser.cpp
│   ├── semantic.cpp
│   ├── ir.cpp
│   ├── optimizer.cpp
│   ├── energy.cpp
│   ├── codegen.cpp
│   ├── report.cpp
│   └── main.cpp
│
├── samples/
│   ├── hello.eco
│   ├── loops.eco
│   └── nested.eco
│
├── tests/
│
├── CMakeLists.txt
└── README.md
```

Work Split (Overall)

Person A – Lexer + Parser

- Design the language (keywords, operators, grammar) — do this together as a team first, it's the shared contract everyone needs.
- Agree on the AST output format as a team (JSON-like structure) — this is A's deliverable interface.
- Build the lexer (source → tokens).
- Build the parser (tokens → AST matching the agreed format).
- Handle lexer/parser errors (bad symbols, missing end, etc.).
- Test against 10-15 sample programs.
- Document grammar + token spec + AST diagram.

Person B – Semantic Analysis + Energy Engine

(builds against hand-written fixture ASTs — never blocked on A)

- Help agree on the AST format and the IR output format — B's deliverable interface.
- Write 3-4 fixture AST files by hand matching the agreed format (e.g. one for let, one for repeat, one with an error).
- Build symbol table + semantic checks against those fixtures.
- Build AST → IR lowering against those fixtures.
- Build the energy cost table + energy calculation engine against hand-written fixture IR too.
- Handle nested loop multiplication, test manually.
- Document the cost model and assumptions.
- Once A's real parser is ready: swap fixture ASTs for A's real output — should be a near drop-in if the format was followed.

Note: Because Energy Analysis runs after Optimization, the energy engine's real input at integration time is Person C's optimized IR, not B's own unoptimized IR. This doesn't block parallel development — B tests the engine against fixture IR either way; only the final pipeline wiring needs to call the optimizer before the energy engine.

Person C – Optimizations + Final Report

(builds against hand-written fixture IR — never blocked on A or B)

- Help agree on the IR format — this is C's input interface.
- Write 3-4 fixture IR lists by hand (a foldable-constant program, a loop with I/O, one with an unused variable).
- Build constant folding + inefficient-loop detection against those fixtures.
- Build the suggestion rule engine against those fixtures.
- Build the final report formatter (matches the mockup layout).
- Build the CLI/pipeline glue code and, optionally, the web UI.
- Once B's real IR + energy engine are ready: swap fixtures for real output.
- Own the final end-to-end integration test and demo prep, since your component sits last.

Project Development Strategy (Parallel Workflow)

Instead of waiting for one person's module to be completed before the next person can start, the team first agrees on the data formats (interfaces) between each compiler stage. Each member then develops their module using hand-written fixture (sample) data, allowing all three members to work simultaneously.

Before writing any code, the team will decide and document the following:

1. Grammar

Define the syntax of the language, including:

- Keywords (let, repeat, print, end)
- Operators (+, -, *, /, =)
- Variables
- Numbers
- Strings
- Statement structure

2. Token Format

Agree on the structure of tokens produced by the lexer.

```
{
  "type": "KEYWORD",
  "value": "repeat",
  "line": 1
}
```

3. AST Format

Agree on the structure of the Abstract Syntax Tree produced by the parser.

```
{
  "type": "RepeatStatement",
  "count": 3,
  "body": [
    {
      "type": "PrintStatement",
      "value": "Hello"
    }
  ]
}
```

4. IR (Intermediate Representation) Format

Agree on the Intermediate Representation that will be generated after semantic analysis.

```
[
  { "op": "LOOP_START", "count": 3 },
  { "op": "PRINT", "value": "Hello" },
  { "op": "LOOP_END" }
]
```

5. Create Fixture Files

Each member creates a few sample Tokens, ASTs, or IRs by hand based on the agreed format. These fixtures will be used for development and testing before integration.

Person A – Lexer & Parser (Compiler Front-End)

Goal: Convert EcoLang source code into an Abstract Syntax Tree (AST).

Step 1: Finalize EcoLang Grammar

- Define the syntax rules of the language.
- Decide the supported statements (let, repeat, print).
- Finalize operators and expressions.

Step 2: Build the Lexer

- Read the source code character by character.
- Convert the code into tokens.
- Identify keywords, identifiers, numbers, strings, and operators.
- Track line numbers for error reporting.

```
Input
let x = 5

Output
KEYWORD(let)
IDENTIFIER(x)
ASSIGN(=)
NUMBER(5)
```

Step 3: Handle Lexical Errors

Detect and report:

- Invalid characters
- Unterminated strings
- Invalid numbers

Provide meaningful error messages with line numbers.

Step 4: Build the Parser

- Read the token stream.
- Verify that the program follows your defined grammar.
- Construct the Abstract Syntax Tree (AST).

```
Program
├── LetStatement
│   ├── x
│   └── 5
└── PrintStatement
    └── x
```

Step 5: Handle Syntax Errors

Detect errors such as:

- Missing end
- Missing operands
- Invalid statement structure

Report the error without crashing the compiler.

Step 6: Test the Front-End

Test using multiple source programs:

- Valid programs
- Invalid syntax
- Missing keywords
- Incorrect operators

Verify that the generated AST matches the agreed format.

Step 7: Documentation

Document:

- EcoLang grammar
- Token specification
- AST structure
- Parsing process

Person B – Semantic Analysis, IR Generation & Energy Analysis

Goal: Analyze the AST, detect semantic errors, generate the Intermediate Representation (IR), and estimate energy consumption of the optimized program.

Step 1: Create Fixture ASTs

Before Person A finishes, manually create several sample ASTs that follow the agreed format.

- Variable declaration
- Print statement
- Repeat loop
- Invalid variable usage

Step 2: Build the Symbol Table

Create a data structure to store declared variables and their types.

```
x        → int
count    → int
message  → string
```

Step 3: Perform Semantic Analysis

Traverse the AST and detect:

- Undeclared variables
- Variable redeclaration
- Invalid operations
- Type mismatches

Report meaningful semantic errors.

Step 4: Convert AST to IR

Transform the AST into a simple Intermediate Representation.

```
STORE x 5
PRINT x
```

The IR should use simple instructions that are easy to analyze.

Step 5: Design the Energy Cost Model

Assign a cost (energy units) to every instruction.

Operation	Energy Units
Assignment	1
Arithmetic	1
Comparison	1
Loop Overhead	2-3
Print	5

These values represent estimated computational cost rather than actual hardware energy.

Step 6: Build the Energy Analysis Engine

Traverse the optimized IR (received from Person C's optimizer) and calculate:

- Total operations
- Energy units
- Estimated Joules
- Breakdown of loop operations, arithmetic operations, and I/O operations

```
repeat 100
    print "Hello"
end

Estimated:
- Loop overhead   = 100
- Print operations = 500
- Total = 600 energy units
```

Step 7: Handle Nested Loops

Correctly calculate execution counts for nested loops by multiplying the loop iterations.

```
repeat 10
    repeat 20
        print "Hi"
    end
end

The print statement executes:
10 × 20 = 200 times
```

Step 8: Test the Energy Engine

Compare compiler-generated energy values with manually calculated values.

Step 9: Documentation

Document:

- Symbol table
- Semantic analysis
- IR structure
- Energy cost model
- Assumptions and limitations

Person C – Optimization, Report Generation & Integration

Goal: Optimize the program, generate recommendations, produce the final report, and integrate the complete compiler pipeline.

Step 1: Create Fixture IR

Before Person B finishes, manually create several sample IR programs.

- Simple assignment
- Large loop
- Constant expressions
- Repeated calculations

Step 2: Implement Constant Folding

Evaluate constant expressions during compilation.

```
Before:
let x = 2 + 3

After:
let x = 5
```

Step 3: Detect Inefficient Code

Analyze the IR to identify:

- Repeated calculations
- Expensive print statements inside loops
- Nested loops
- Unused variables

Step 4: Generate Optimization Suggestions

Produce recommendations such as:

- Reduce print statements inside loops.
- Cache repeated calculations.
- Remove unused variables.
- Move loop-invariant calculations outside loops.

Step 5: Generate the Final Report

Display the compiler results in a user-friendly format, after energy analysis has run on the optimized IR.

```
Compilation Successful
------------------------------
Energy Analysis Report
```

```
Estimated Operations   : 2003
Estimated Energy Units : 6003
Estimated Energy       : 0.0006 J
Time Complexity        : O(n)

Optimization Suggestions
✓ Reduce print statements inside loops
✓ Cache repeated calculations
✓ Remove unused variables
```

Step 6: Build the Compiler Driver

Create the main program that connects all compiler stages.

```
EcoLang Source Code
        ↓
Lexer
        ↓
Parser
        ↓
Semantic Analysis
        ↓
IR Generation
        ↓
Optimization
        ↓
Energy Analysis
        ↓
Code Generation
        ↓
Final Report
```

Note: This driver order places Optimization before Energy Analysis and adds Code Generation before the Final Report — the original draft's driver pipeline skipped Code Generation and had Optimization after Energy Analysis.

Step 7: Integration

Replace fixture IRs with the real IR generated by Person B and verify that the complete compiler pipeline works correctly.

Step 8: End-to-End Testing

Test the complete compiler using multiple EcoLang programs, including:

- Valid programs
- Semantic errors
- Large loops
- Nested loops
- Programs with optimization opportunities

Step 9: Demo Preparation

Prepare sample programs that demonstrate:

- Successful compilation
- Energy estimation
- Semantic error detection
- Optimization suggestions

- Final energy analysis report

Project Timeline

Day 1

- Finalize EcoLang grammar.
- Finalize Token, AST, and IR formats.
- Create fixture files.

Development Phase (Parallel Work)

- Person A: Lexer, Parser, AST generation.
- Person B: Semantic analysis, Symbol table, IR generation, Energy analysis.
- Person C: Optimizations, Suggestions, Report generation, Compiler integration.

Integration Phase

- Replace fixture data with actual outputs.
- Connect all compiler modules in the corrected order (Optimization before Energy Analysis).
- Resolve interface mismatches.
- Perform end-to-end testing.

Final Phase

- Prepare documentation.
- Test all sample programs.
- Refine the compiler output.
- Practice the final project demonstration.